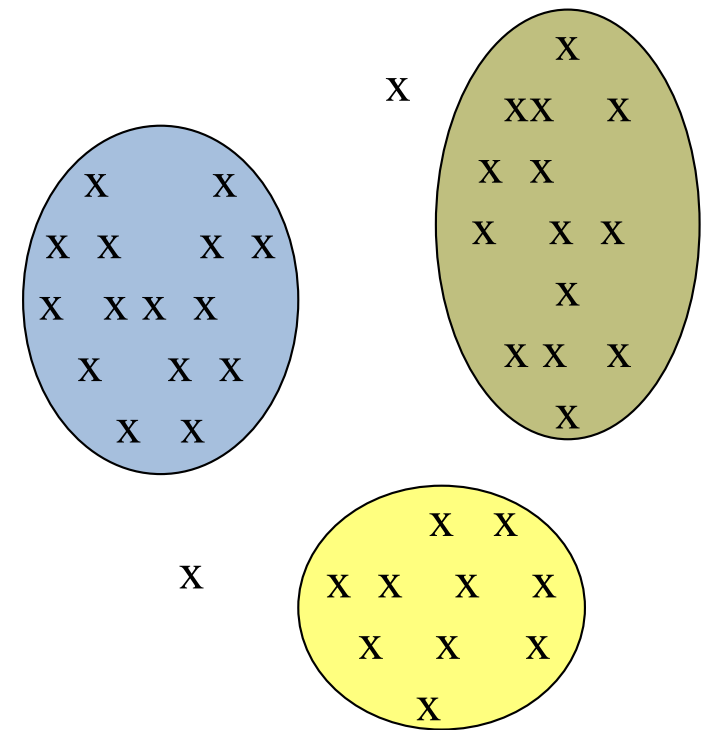




Clustering Analysis (1)

What Is Clustering Analysis?

- Clustering analysis.
 - Grouping a set of data objects into clusters, such that objects within each cluster are similar to each other, objects in different clusters are dissimilar to each other.
- Clustering is **unsupervised** classification:
 - Objects are not labeled with predefined classes.
 - Different from **supervised** classification where each training data is labeled with class information

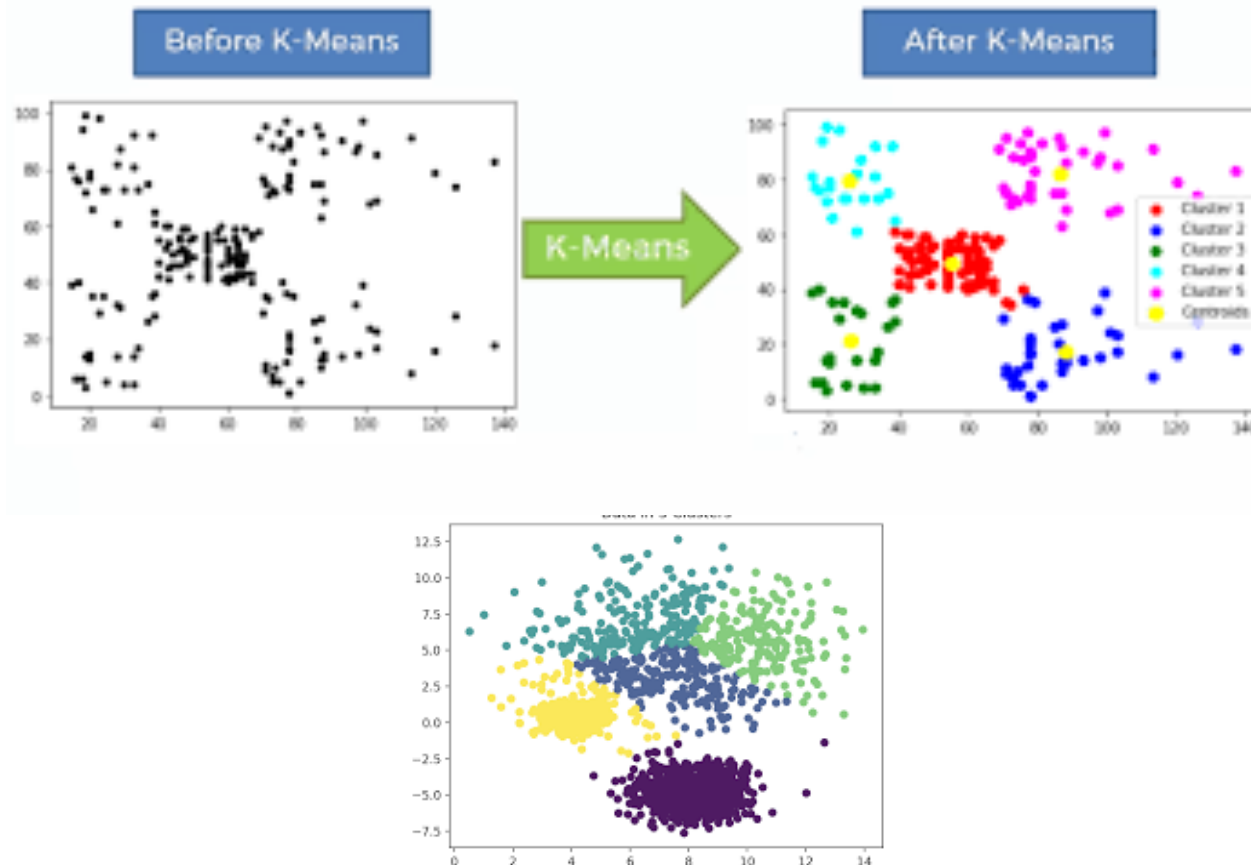


General Applications

- Typical applications.
 - As a stand-alone tool to get insight into data distribution.
 - As a preprocessing step for other algorithms.
- Examples:
 - (Spatial) data analysis
 - Image processing
 - Economic science (especially market research)
 - WWW
 - Automatic document categorization
 - Web usage mining: cluster web log data to discover groups of similar access patterns
 - Business : customer groups
 - Biology: animal and plant taxonomy, Categorize genes by functionality

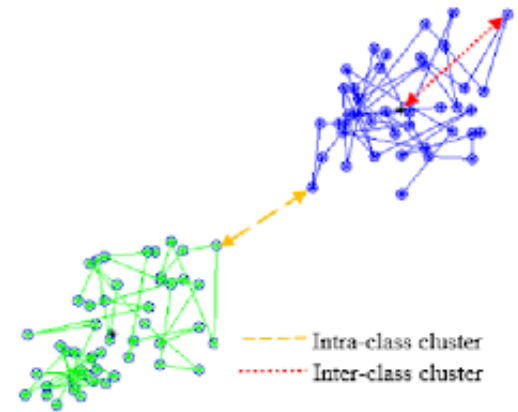
Clustering Intuition

- What does it mean to group similar points?
- How do we decide similarity?



What Is Good Clustering?

- A good clustering method will produce high quality clusters with.
 - High intra-class similarity.
 - Low inter-class similarity.
- The quality of a clustering result depends on both the dissimilarity (or distance) measure used by the method and its clustering approach used.
- The quality of a clustering method is also measured by its ability to discover some or all of the *hidden* patterns



How to compute distance?

- Data
$$\begin{bmatrix} x_{11} & \dots & x_{1f} & \dots & x_{1p} \\ \dots & \dots & \dots & \dots & \dots \\ x_{i1} & \dots & x_{if} & \dots & x_{ip} \\ \dots & \dots & \dots & \dots & \dots \\ x_{n1} & \dots & x_{nf} & \dots & x_{np} \end{bmatrix}$$

- Distance Measures

- Euclidean Distance

- **Straight-line distance** between two points

$$d(i, j) = \sqrt{\sum_{k=1}^p (x_{ik} - x_{jk})^2}$$

- Manhattan Distance

- Distance along **grid paths**

$$d(i, j) = \sum_{k=1}^p |x_{ik} - x_{jk}|$$

The *K-Means* Clustering

- **Objective:** to form a set of clusters that are as compact and separated as possible
- **Distance Measure:**
 - Euclidean distance between data object and cluster center
- **Clustering criterion function:**
mean squared error (MSE)

$$MSE = \sum_{i=1}^k \sum_{p \in C_i} |x - m_i|^2$$

x: a data object

C_i: cluster *i*

m_i: center of cluster *i*

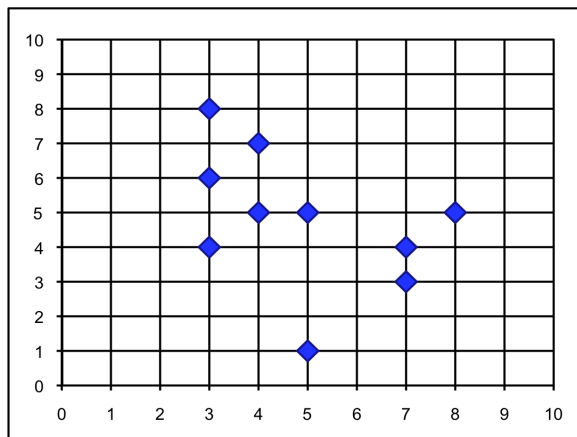
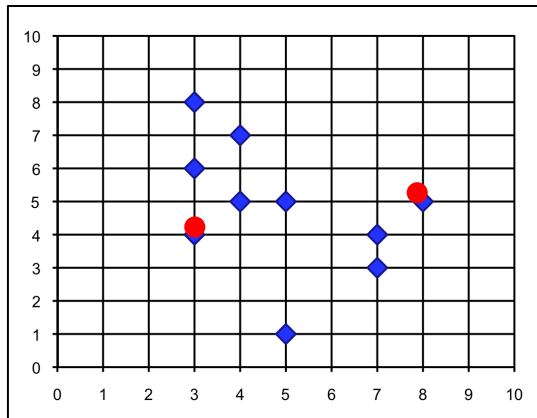
k: number of clusters

The *K-Means* Clustering

K-Means Clustering Algorithm

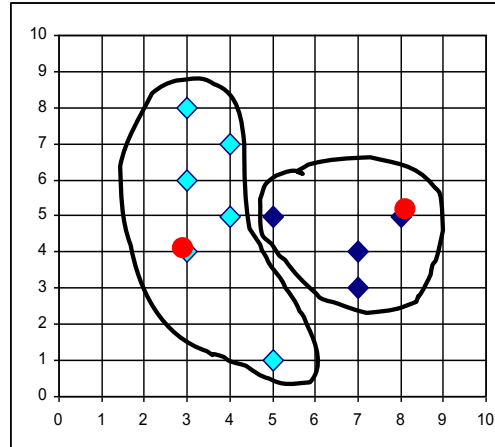
- Given K , choose K objects as the initial cluster centers
- **Repeat**
 - Compute centroids (mean points) of the current clusters
 - Assign each object to the cluster with the nearest centroid
- **Until**
 - no change in cluster assignment, or
 - the clustering criterion (mean squared error) converges

K-Means

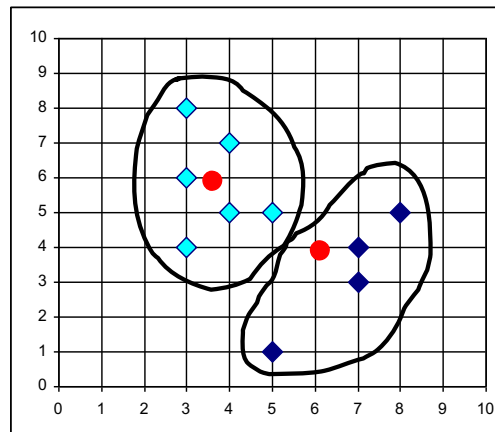


$K=2$, Arbitrarily choose K object as initial cluster center

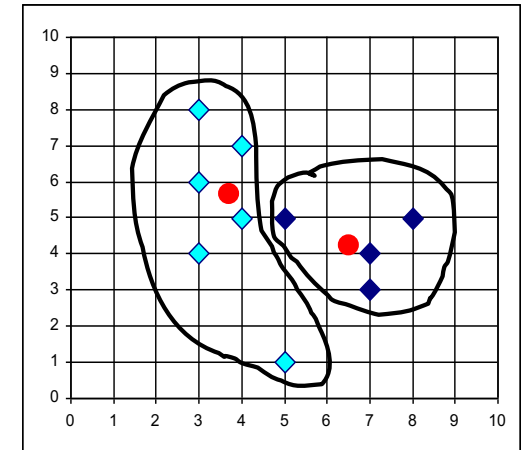
Assign each object to most similar center



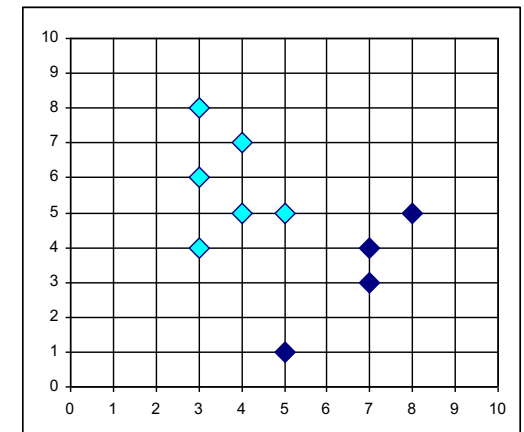
reassign



Update the cluster centers



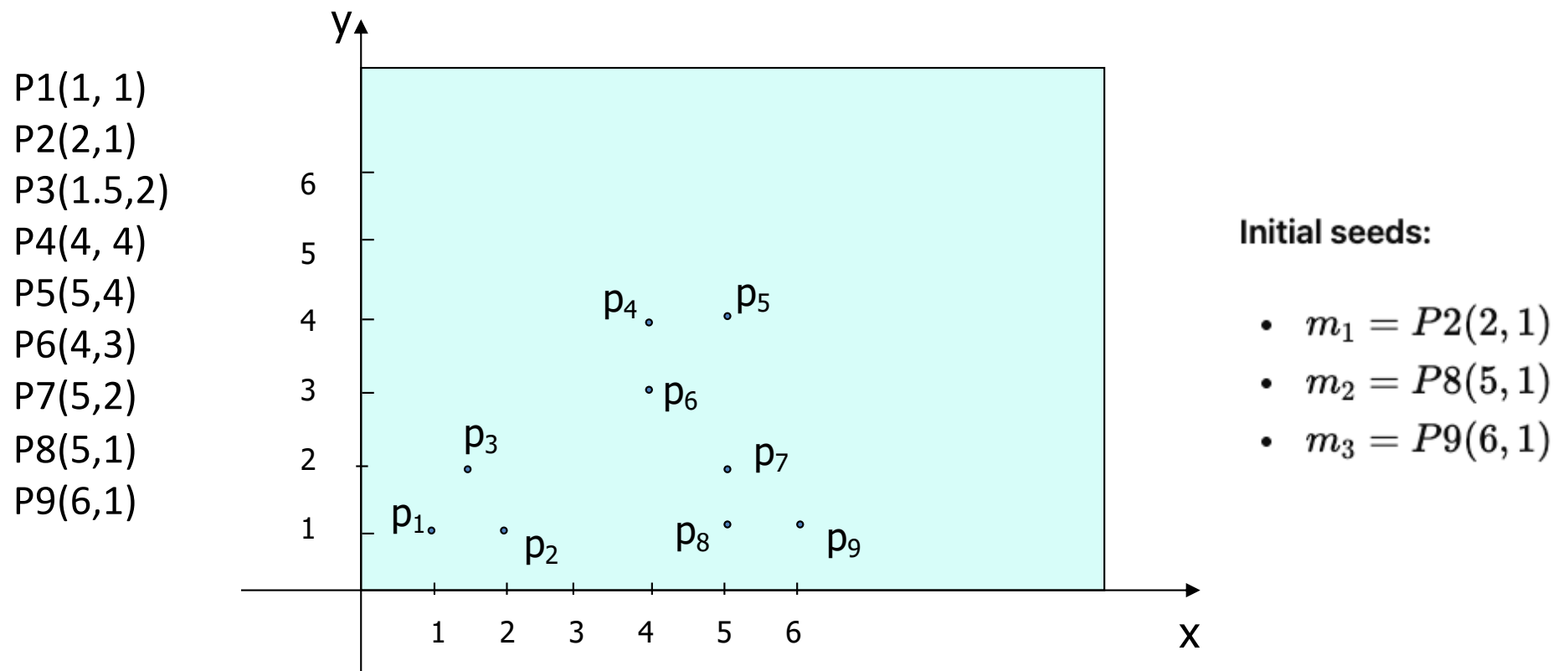
reassign



Update the cluster centers

Practice Question

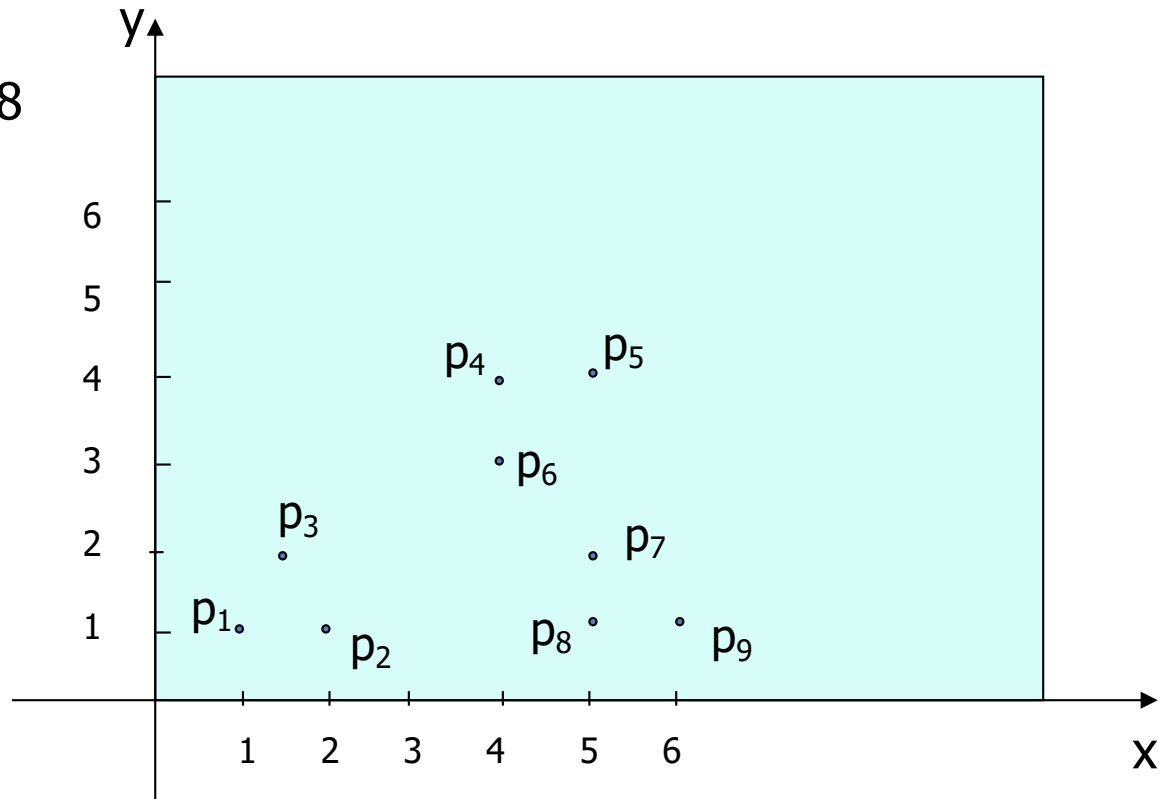
Apply K-means clustering algorithm to partition the following data with 9 data objects:



Iteration 1 (Assignment)

**Assign points to nearest centroid
(based on the distance measure)**

- **Cluster 1:** P1, P2, P3
- **Cluster 2:** P4, P5, P6, P7, P8
- **Cluster 3:** P9



Iteration 1 (Update)

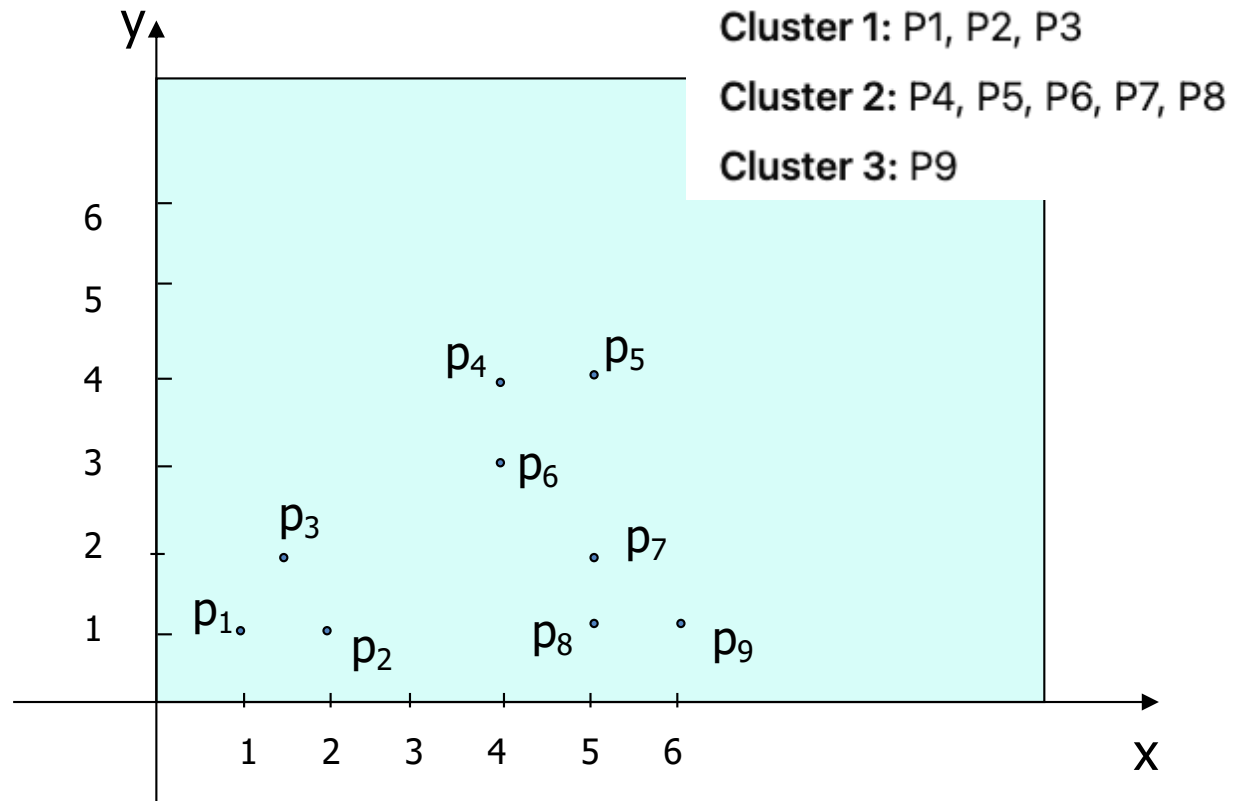
Update centroids

$$m_1 = (1.5, 1.33)$$

$$m_2 = (4.6, 2.8)$$

$$m_3 = (6, 1)$$

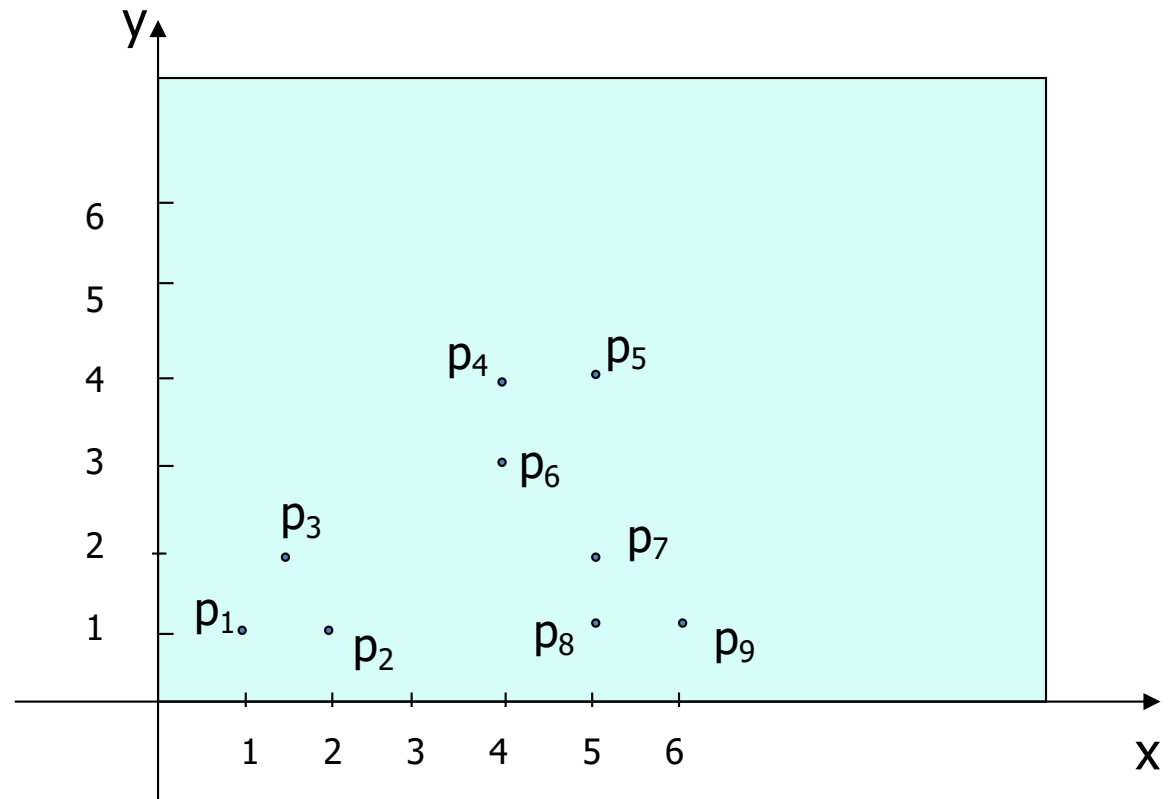
$$m_2 = \left(\frac{\sum x_i}{5}, \frac{\sum y_i}{5} \right) = (4.6, 2.8)$$



Iteration 2 (Assignment)

Reassign points

- Cluster 1: P1, P2, P3
- Cluster 2: P4, P5, P6, P7
- Cluster 3: P8, P9



P8 moves to Cluster 3

$$\begin{aligned} d(P8, m_2) &= \sqrt{(5 - 4.6)^2 + (1 - 2.8)^2} \\ &= \sqrt{(0.4)^2 + (-1.8)^2} = \sqrt{0.16 + 3.24} = \sqrt{3.40} \approx 1.844 \end{aligned}$$

$$\begin{aligned} d(P8, m_3) &= \sqrt{(5 - 6)^2 + (1 - 1)^2} \\ &= \sqrt{(-1)^2 + 0^2} = \sqrt{1} = 1 \end{aligned}$$



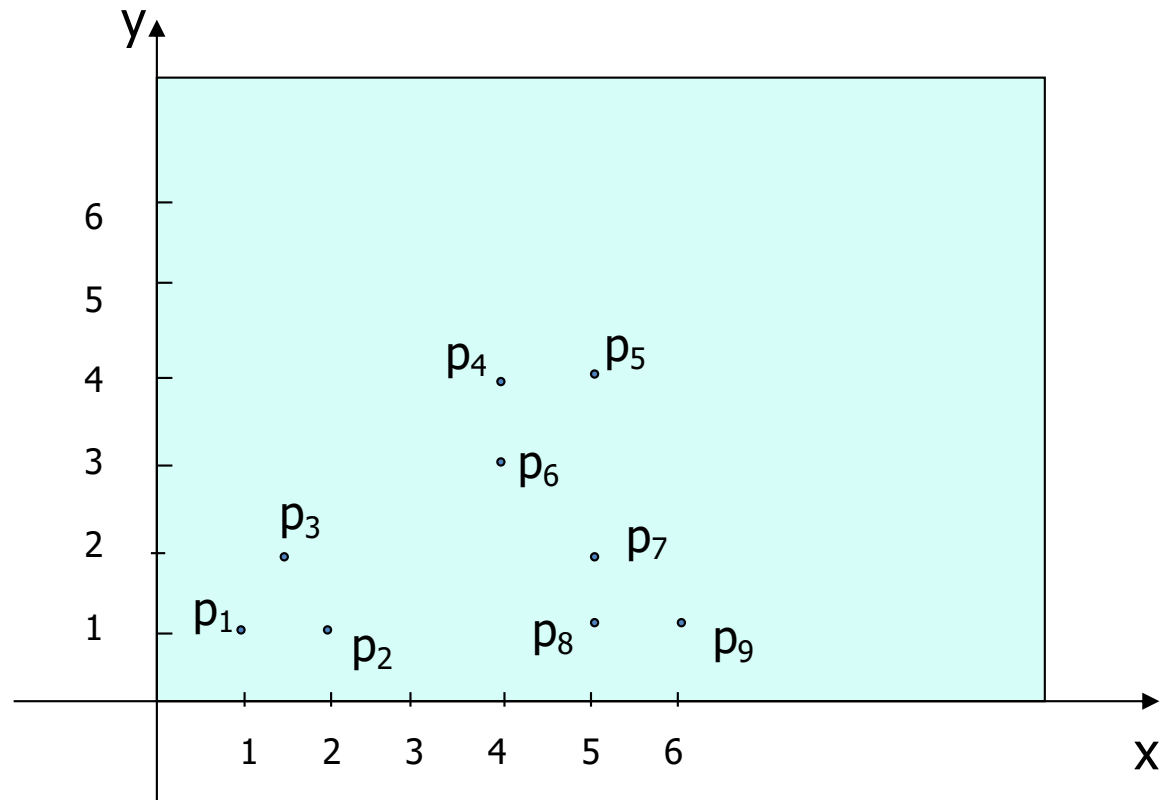
Iteration 2 (Update)

Update centroids

$$m_1 = (1.5, 1.33)$$

$$m_2 = (4.5, 3.25)$$

$$m_3 = (5.5, 1)$$



Iteration 3 (Assignment)

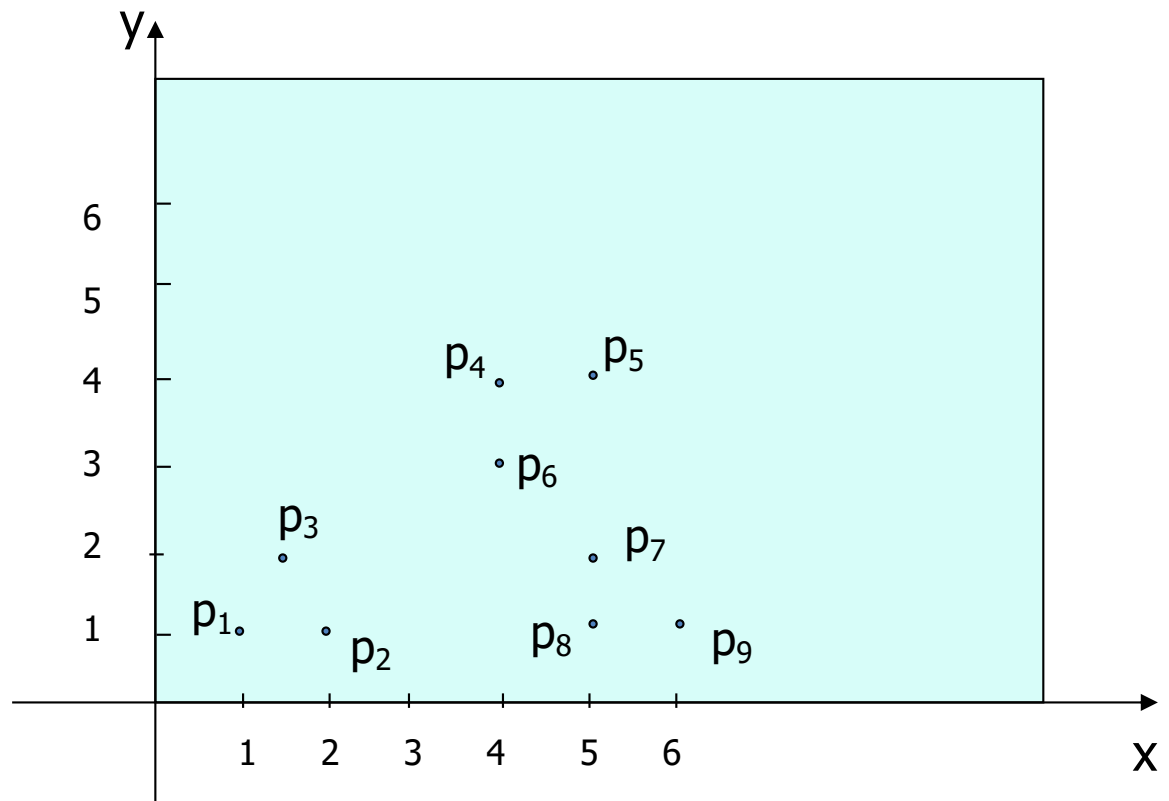
Reassign points

- Cluster 1: P1, P2, P3
- Cluster 2: P4, P5, P6
- Cluster 3: P7, P8, P9

P7 moves to Cluster 3

$$d(P7, m_2) = 1.346$$

$$d(P7, m_3) = 1.25$$



Convergence

No change in assignments → Converged

Final clusters:

- C1: P1, P2, P3
- C2: P4, P5, P6
- C3: P7, P8, P9

Mean Squared Error (MSE)

$$MSE = \sum_{i=1}^k \sum_{p \in C_i} \|p - m_i\|^2$$

$$MSE = 3.8544$$

K-Means: Strengths & Limitations

Strengths

- **Fast and scalable**
 - $O(tkn)$, $k, t \ll n$
- **Simple and easy to implement**
 - intuitive “assign + update” process
- **Works well for compact, spherical clusters**

K-Means: Strengths & Limitations

Limitations

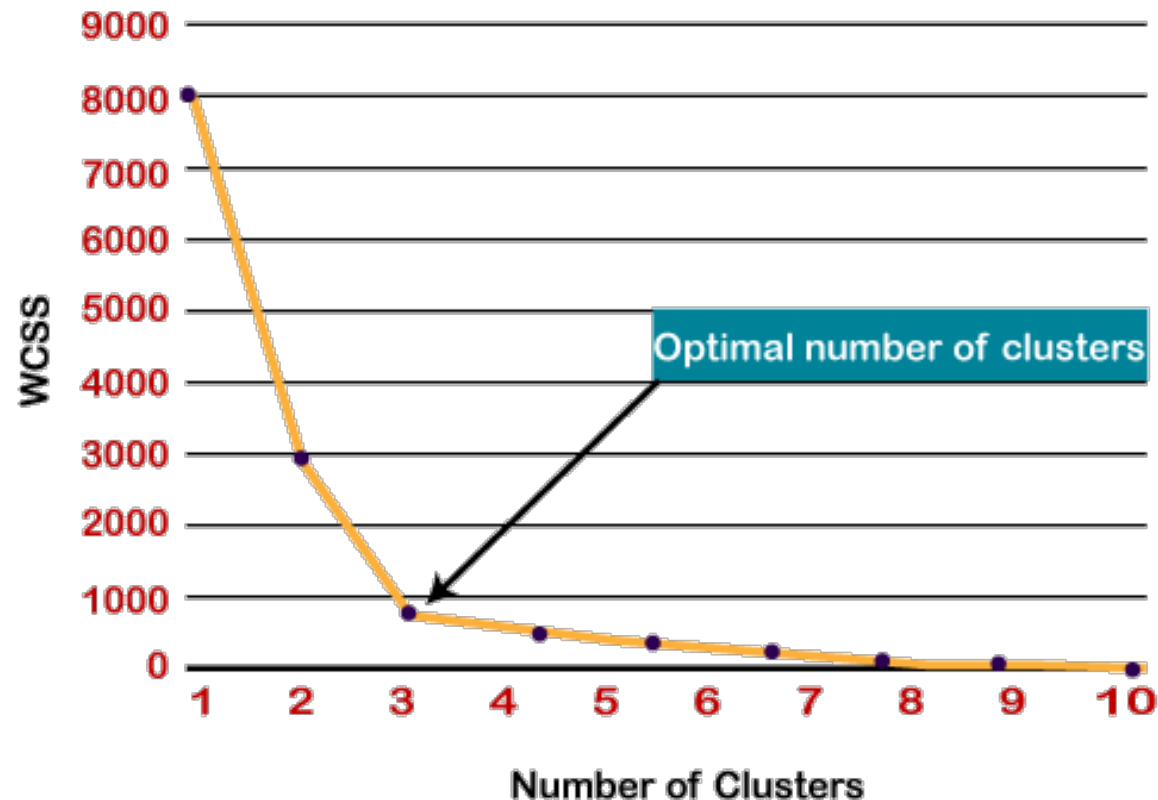
- **Requires numeric data**
 - mean is not defined for many other types of data, e.g., categorical and ordered
- **Must choose k in advance**
 - result depends on this choice
- **Sensitive to initialization**
 - may converge to different (local) optima
- **Sensitive to outliers and noise**
 - centroids can be pulled away

Beyond K-Means: What Comes Next

- **Choosing k**
 - Elbow method, Silhouette score
- **Distance matters**
 - Different data types → different similarity measures
- **Numeric data only**
 - k-medoids (handles general distance measures)
- **Sensitive to outliers / noise**
 - k-medoids (more robust), DBSCAN (later)
- **Scalability (large datasets)**
 - Mini-batch K-means

Deciding on the optimal number of clusters

The Elbow Method



$$WCSS = \sum_{k=1}^K \sum_{x \in C_k} ||x - \mu_k||^2$$

C_k = cluster k

μ_k = centroid of cluster k

$||x - \mu_k||^2$ = squared distance

Silhouette Score

- Measures how well each point fits within its cluster
- **For each point:**
 - a : average distance to points in the **same cluster**
 - b : average distance to points in the **nearest other cluster**

$$s = \frac{b - a}{\max(a, b)}$$

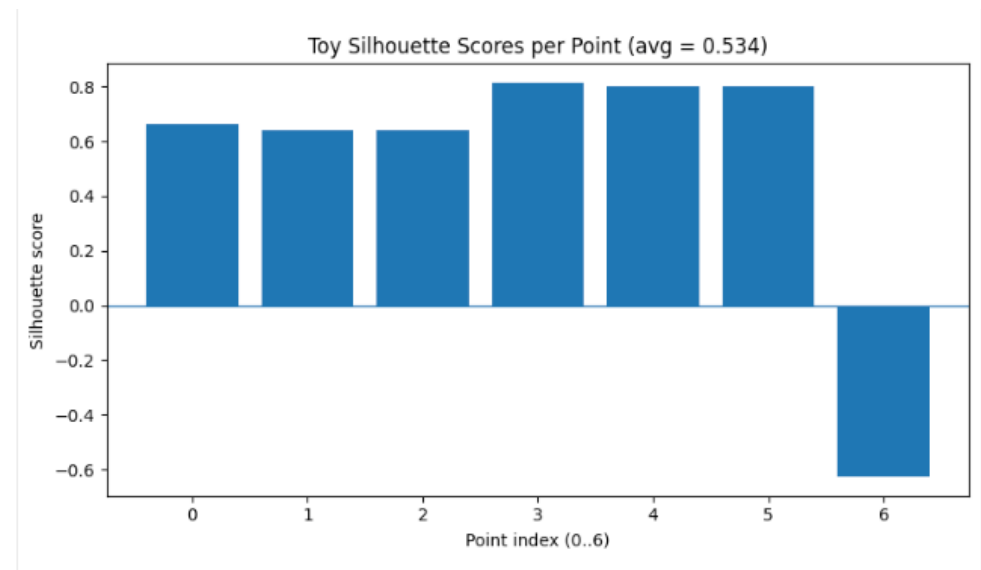
$s \approx 1 \rightarrow$ well clustered

$s \approx 0 \rightarrow$ on boundary

$s < 0 \rightarrow$ likely misclassified

Average s over all points

Higher is better



Mini-Batch K-Means (Idea)

- Approximate K-means using partial data
- Designed for **large datasets**
- **Approach:**

Initialize K centroids

Repeat:

- Sample a mini-batch of data points
 - Assign points to nearest centroid
 - Update centroids using only this batch
- The current centroids are carried forward and updated with each new batch $\mu_k \leftarrow (1 - \eta)\mu_k + \eta x$
 - In sklearn: **MiniBatchKMeans**

Types of Data in Clustering Analysis

- Numeric variables (Interval-scaled)
- Binary variables
- Nominal, and ordinal variables
- Variables of mixed types
- Text
- Temporal

Similarity/Dissimilarity for Binary Data

- A contingency table for binary data

		Object j		
		1	0	sum
Object i	1	a	b	$a+b$
	0	c	d	$c+d$
	sum	$a+c$	$b+d$	p

- Simple matching coefficient (if the binary variable is symmetric):

$$d(i, j) = \frac{b+c}{a+b+c+d}$$

- Jaccard coefficient (if the binary variable is asymmetric):

$$d(i, j) = \frac{b+c}{a+b+c}$$

Dissimilarity between Binary Variables

- Example

Name	Gender	Fever	Cough	Test-1	Test-2	Test-3	Test-4
Jack	M	Y	N	P	N	N	N
Mary	F	Y	N	P	N	P	N
Jim	M	Y	P	N	N	N	N

- gender is a symmetric attribute
- the remaining attributes are asymmetric binary
- let the values Y and P be set to 1, and the value N be set to 0

Jaccard
coefficient

$$d(\text{jack}, \text{mary}) = \frac{0 + 1}{2 + 0 + 1} = 0.33$$

$$d(\text{jack}, \text{jim}) = \frac{1 + 1}{1 + 1 + 1} = 0.67$$

$$d(\text{jim}, \text{mary}) = \frac{1 + 2}{1 + 1 + 2} = 0.75$$

Nominal Attributes

- A generalization of the binary attribute in that it can take more than 2 states, e.g., red, yellow, blue, green
- Method 1: Simple matching
 - m : # of matches, p : total # of variables
$$d(i, j) = \frac{p - m}{p}$$
- Method 2: use a large number of binary attributes
 - creating a new binary variable for each of the M nominal states

Ordinal Attributes

- An ordinal attribute can be discrete or continuous
- order is important, e.g., rank
- Can be treated like interval-scaled
 - replacing x_{if} by their rank $r_{if} \in \{1, \dots, M_f\}$
 - map the range of each attribute onto $[0, 1]$ by replacing i -th object in the f -th attribute by
$$z_{if} = \frac{r_{if} - 1}{M_f - 1}$$
 - compute the dissimilarity using methods for interval-scaled attributes

Attributes of Mixed Types

- A database may contain different types of attributes
 - symmetric binary, asymmetric binary, nominal, ordinal, and interval.

Other Similarity/Distance measures

- **Sets as vectors:** measure similarity by the cosine distance.

$$x_i = [x_{i1}, x_{i2}, \dots, x_{ip}]$$

$$x_j = [x_{j1}, x_{j2}, \dots, x_{jp}]$$

$$\cos(x_i, x_j) = \frac{x_i \bullet x_j}{|x_i| * |x_j|} = \hat{x}_i \bullet \hat{x}_j$$

Other Similarity/Distance measures

- Measure distance between words/address/query, or between DNA sequences by edit distance
 - Given two strings S_1 and S_2 , the minimum number of operations to convert one to the other
 - Operations are typically character-level
 - Insert, Delete, Replace, (Transposition)
 - E.g., the edit distance from **dof** to **dog** is 1
 - From **cat** to **act** is 2 (Just 1 with transpose.)
 - from **cat** to **dog** is 3.
- Generally found by dynamic programming.

Attributes of Mixed Types

- Use weighted formula to combine their effects.

- Feature value missing, or asymmetric binary with

$$x_{if} = x_{jf} = 0 \quad \rightarrow \quad \delta_{ij}(f) = 0$$

- Otherwise $\rightarrow \delta_{ij}(f) = 1$

- feature is interval-based: use the normalized distance ($\delta_{ij}^{(f)}$ is weight on feature f)

$$d(i, j) = \frac{\sum_{f=1}^P \delta_{ij}^{(f)} d_{ij}^{(f)}}{\sum_{f=1}^P \delta_{ij}^{(f)}}$$

- feature is ordinal

- compute ranks r_{if} and
- and treat z_{if} as interval-scaled

$$z_{if} = \frac{r_{if} - 1}{M_f - 1}$$

Mixed-Type Distance Example (with Standardization)

- Compute the distance between (obj1, obj2),

	Gender	Age	Heart Rate	Fever	Cough	Category
Obj1	F	18	120	Y	N	Severe-1
Obj2	M	36	89	N	N	Normal

Age: $\mu_{age} = 42, \sigma_{age} = 3.5$

Heart Rate: $\mu_{hr} = 95, \sigma_{hr} = 10$

Possible values for Category include : Normal, Severe-1, Severe-2, Dying
For simplicity in demonstration, use Manhattan distance for interval data.

Mixed-Type Distance Example (with Standardization)

Age:

$$z_1 = \frac{18 - 42}{3.5} = -6.85, \quad z_2 = \frac{36 - 42}{3.5} = -1.71$$

$$d_{\text{age}} = |-6.85 - (-1.71)| = 5.14$$

Heart Rate:

$$z_1 = \frac{120 - 95}{10} = 2.5, \quad z_2 = \frac{89 - 95}{10} = -0.6$$

$$d_{\text{hr}} = |2.5 - (-0.6)| = 3.1$$

Category: normal(1), severe1(2), severe2(3), dying(4)

$$d_{\text{cat}} = \frac{|2 - 1|}{4 - 1} = \frac{1}{3} = 0.33$$

Mixed-Type Distance Example (with Standardization)

Fever + Cough (asymmetric binary)

Obj1: fever = 1, cough = 0

Obj2: fever = 0, cough = 0

$$d_{\text{bin}} = \frac{b + c}{a + b + c} = \frac{1 + 0}{0 + 1 + 0} = 1$$

Overall distance/dissimilarity, Fever+Cough as one binary block:

$$D = \frac{d_{\text{age}} + d_{\text{hr}} + d_{\text{cat}} + d_{\text{bin}}}{4} = \frac{5.14 + 3.1 + 0.33 + 1}{4} = \frac{9.57}{4} = 2.3925$$

Overall distance/dissimilarity, Fever+Cough as separate features:

$$D = \frac{5.14 + 3.1 + 0.33 + 1 + 0}{5} = \frac{9.57}{5} = 1.914$$

Major Clustering Approaches

- Partitioning algorithms: Construct various partitions and then evaluate them by some criterion
- Hierarchy algorithms: Create a hierarchical decomposition of the set of data (or objects) using some criterion
- Model-based: A model is hypothesized for each of the clusters and the idea is to find the best fit of that model to each other

(Compare the clusters discovered by different types of clustering systems:

<https://scikit-learn.org/stable/modules/clustering.html>)

Partitioning Algorithms: Basic Concept

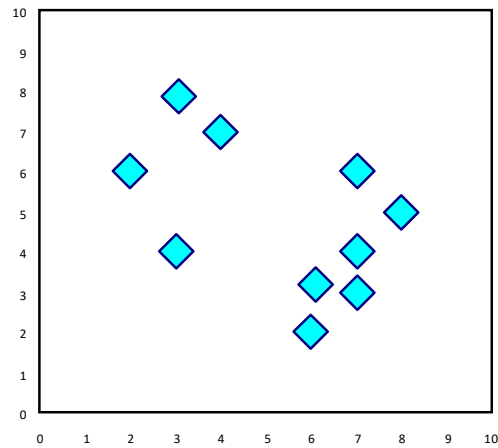
- Partitioning method: Construct a partition of a database ***D*** of ***n*** objects into a set of ***k*** clusters
- Given a ***k***, find a partition of ***k clusters*** that optimizes the chosen partitioning criterion
 - Global optimal: exhaustively enumerate all partitions
 - Heuristic methods: *k-means* and *k-medoids* algorithms
 - *k-means* (MacQueen' 67): Each cluster is represented by the center of the cluster
 - *k-medoids* or PAM (Partition around medoids) (Kaufman & Rousseeuw' 87): Each cluster is represented by one of the objects in the cluster

K-Medoids

- Arbitrarily choose K objects as the initial medoids, O_j s;
- Repeat:
 - Assign each remaining object to the cluster with the nearest medoids;
 - Randomly select a nonmedoid object O_{random} ;
 - Compute the total cost, S , of swapping O_j with O_{random} ;
 - If S decreases, then swap O_j with O_{random} to form the new set of k medoids;
- Until no change

k-Medoids

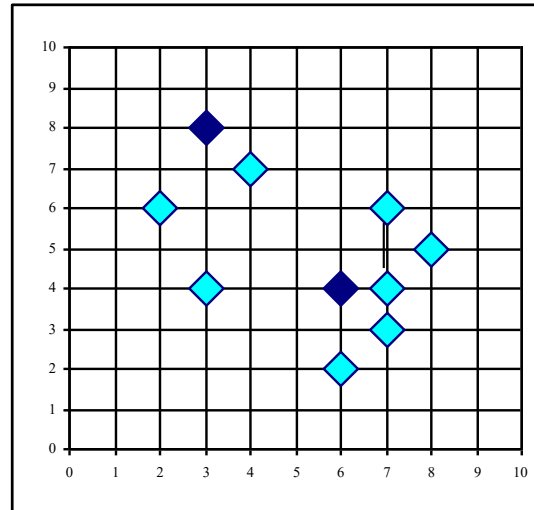
Total swapping cost $TC_{ih} = \sum_p C_{pih}$



K=2

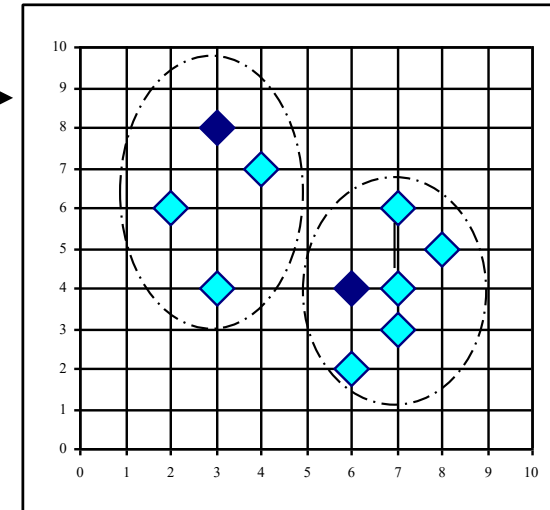
Do loop
Until no
change

Arbitrary
choose k
object as
initial
medoids



Total Cost = 18

Assign
each
remaining
object to
nearest
medoids

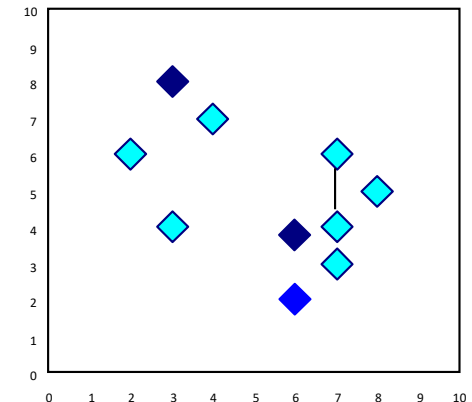
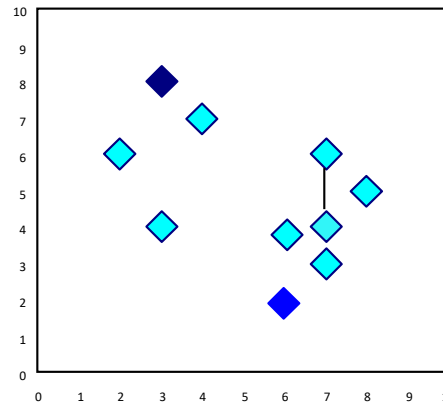


Total Cost = 20

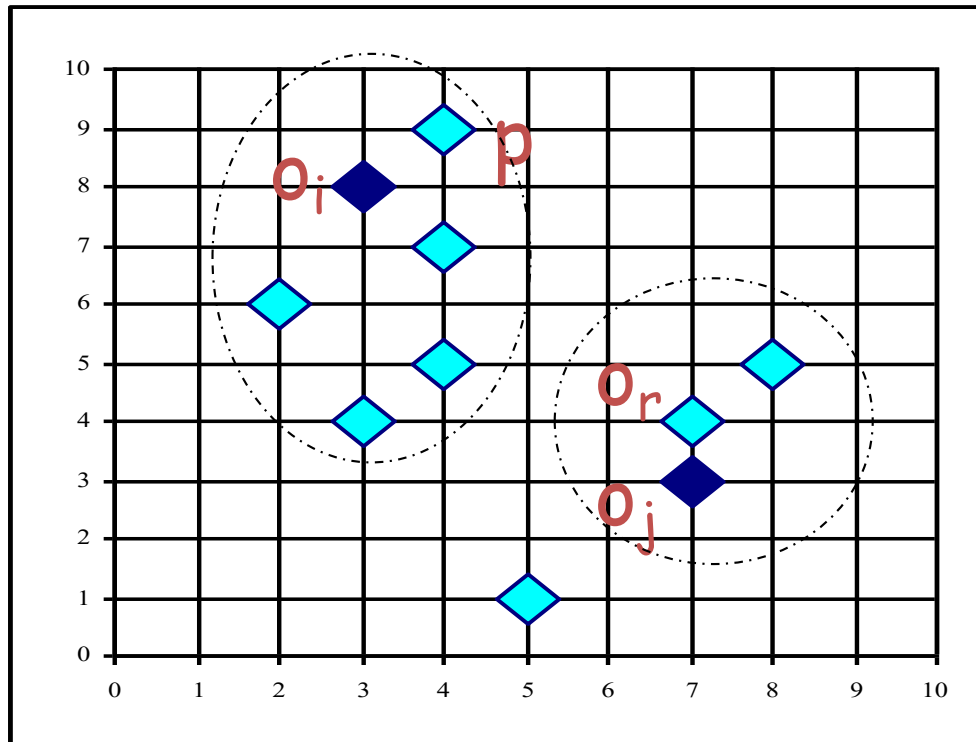
Randomly select a
nonmedoid object, O_{random}

Compute
total cost of
swapping

Swapping O
and O_{random}
if quality is
improved.



Four Cases – Case A



Replace o_j with o_r

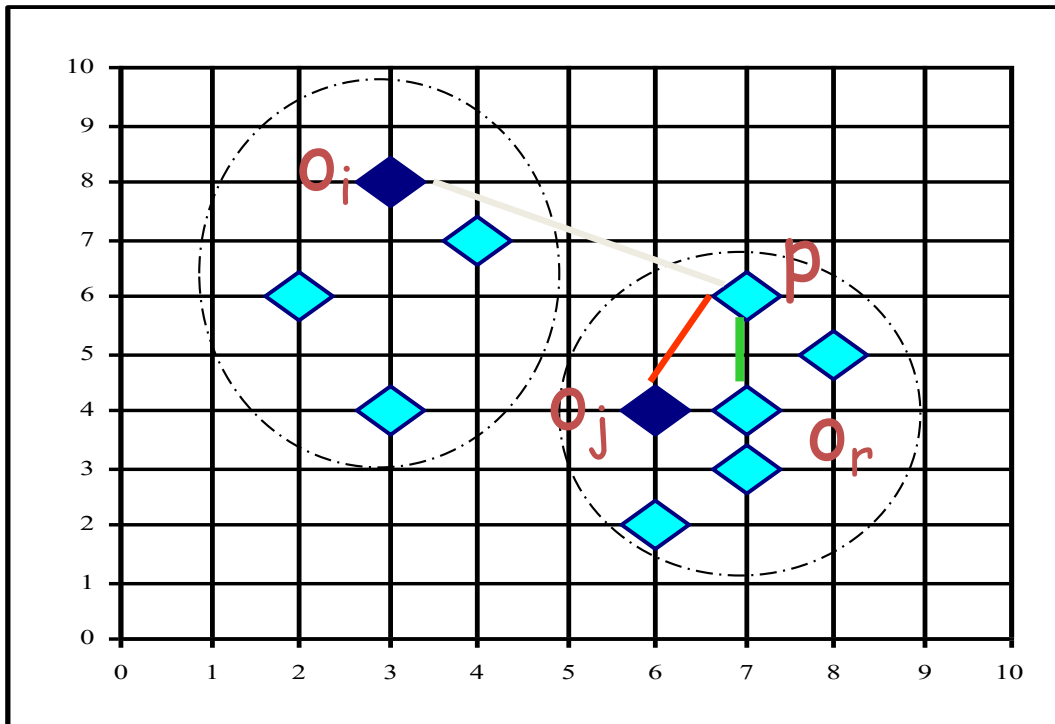
$p \in o_i ; i \neq j;$

p still closest to o_i

no change

$$C_{p,j,r} = 0$$

Four Cases – Case B



Replace o_j with o_r

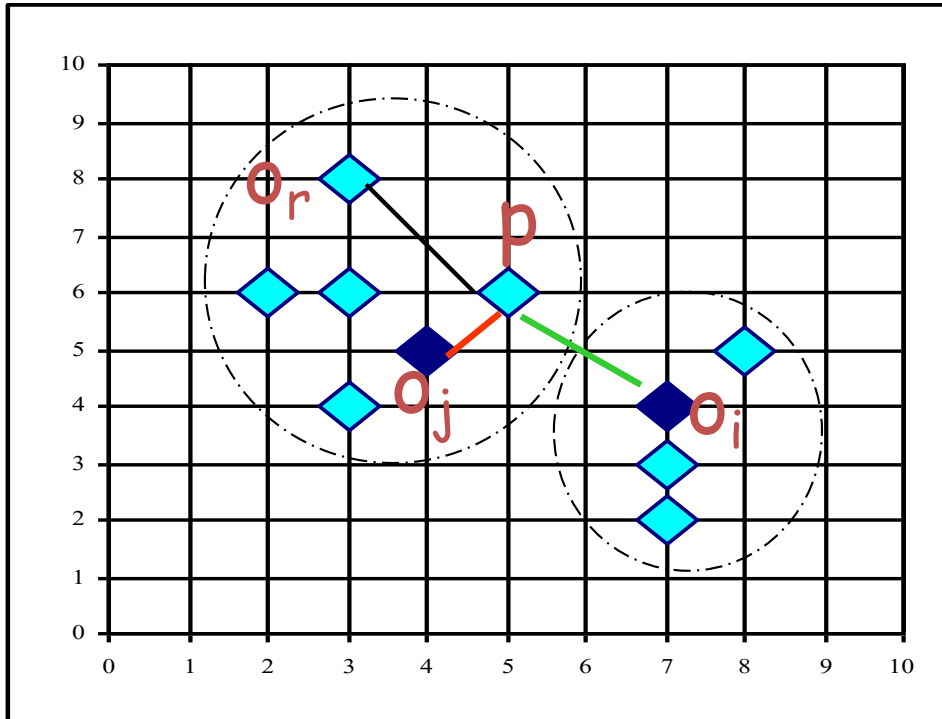
$p \in o_j$

p closest to o_r

Reassign p to O_r

$$C_{p,j,r} = d(p - o_r) - d(p - o_j)$$

Four Cases – Case C



Replace o_j with o_r

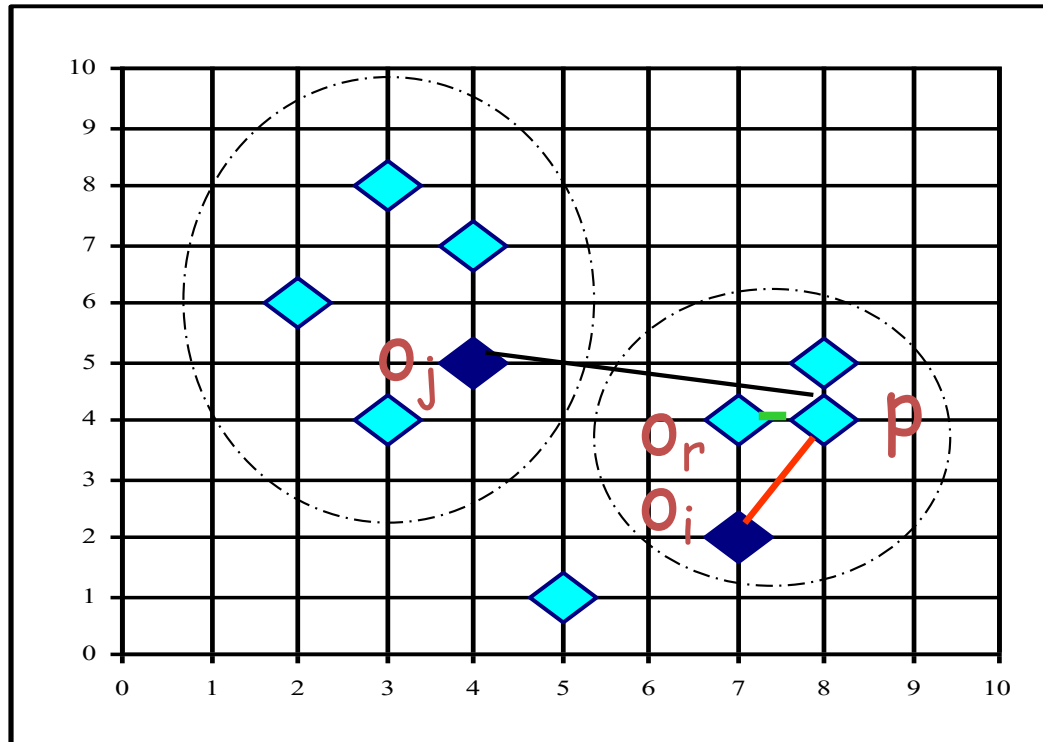
$$p \in o_j$$

p is now closer to o_i $i \neq j$

Reassign p to O_i

$$C_{p,j,r} = d(p - o_i) - d(p - o_j)$$

Four Cases – Case D



Replace o_j with o_r

$p \in o_i ; i \neq j;$

p closest to o_r

Reassign p to O_r

$$C_{p,j,r} = d(p - o_r) - d(p - o_i)$$

PAM Complexity Analysis

- Total $k*(n-k)$ pairs of (O_i, O_r) , k is the number of clusters
- For each pair of (O_i, O_r) :
 - compute Tc_{ir} require the examination of $(n-k)$ non-selected objects.
- Total complexity:
 $O(k*(n-k)^2)$

Compare K-means and PAM

- K-means is computationally more efficient
- K-means only handles numeric data
- PAM can handle different types of data
- PAM is better in terms of handling outliers in data

The CLARA algorithm

- Objective: to improve the computational efficiency of PAM, through sampling
- Basic idea:
 - draw a sample (size= $40+2k$) from the original data set, apply PAM on the sample, and finds the medoids of the sample.
 - Repeat the process a fixed number of times and return the set of medoids that generate the lowest average dissimilarity from the entire data set of objects
- Complexity: $O(k*(40+k)^2 + k*(n-k))$

The CLARA Algorithm

for $i=1$ to 5, repeat the following steps:

- Draw a sample of $40+2k$ objects randomly from the entire data set, and call algorithm PAM to find the k medoids of the sample
- For each object O_j in the entire data set, determine which of the k medoids is the most similar to O_j .
- Calculate the average dissimilarity of the clustering obtained in the previous step. If this value is $<$ current minimum, set current minimum to this value, and retain the current set of k medoids
- Return to step 1 to start the next iteration

CLARANS

(“Randomized” CLARA)

- *CLARANS* (A Clustering Algorithm based on Randomized Search)
- CLARANS draws sample of *neighbors* dynamically
- The clustering process can be presented as searching a graph where every node is a potential solution, that is, a set of k medoids
- If the local optimum is found, *CLARANS* starts with new randomly selected node in search for a new local optimum
- It is more efficient and scalable than both *PAM* and *CLARA*

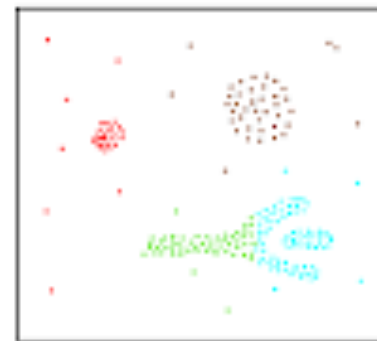
Clusters Discovered by CLARANS



database 1



database 2



database 3

The CLARANS Algorithm

1. Input *numlocal* and *maxneighbor*
 i=1, *mincost*=FLT_MAX, *bestnode*=NULL
2. *current* = an arbitrary *k* modiods
3. *j*=1
4. Pick random neighbor *S* of *current*, compute the cost difference between *S* and *current*
5. If *S* has lower cost, set *current* = *S*, goto 3
 else
 j=*j*+1;
 if (*j* <=*maxneighbor*) goto 4
 else
 if (*cost(current)* < *mincost*)
 mincost = *cost(current)*
 bestnode = *current*
6. *i*= *i*+1;
7. If (*i* <= *numlocal*)
 goto step 2
 else
 output *bestnode* and halt

The CLARANS Algorithm

